

POSTECH



## TECH CHALLENGE

Tech Challenge é o projeto que engloba os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, em princípio, deve ser desenvolvida em grupo. Importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 90% da nota de todas as disciplinas da fase.

### Importante: Projeto Evolutivo e Cumulativo

Este Tech Challenge é a **conclusão** da sua jornada com o projeto **ToggleMaster**. Para realizar esta fase, **é obrigatório que toda a base das Fases 1, 2 e 3 esteja funcionando**.

Isso significa que seu projeto deve conter:

- Os 5 microsserviços containerizados.
- A infraestrutura provisionada via código (Terraform).
- Pipelines de CI/CD (GitHub Actions ou similar) com DevSecOps.
- Implantação no Kubernetes (EKS, GKE, AKS) gerenciada via GitOps (ArgoCD/Flux).

O monitoramento que você construirá agora será aplicado **em cima** dessa arquitetura automatizada.

## DESAFIO

A DevOps Solutions Inc. atingiu o estado da arte na entrega de software. O ToggleMaster é atualizado dezenas de vezes por dia através de pipelines automatizados de CI/CD e GitOps. A infraestrutura é criada em minutos via Terraform.

Porém, um novo problema surgiu no último final de semana: o serviço de avaliação (evaluation-service) começou a falhar silenciosamente. Como a infraestrutura escalava automaticamente (HPA), o consumo de recursos aumentou vertiginosamente para tentar compensar as falhas. A equipe de

engenharia demorou 6 horas para ser avisada pelos usuários e mais 4 horas para achar a causa raiz nos logs espalhados pelos pods.

**O novo objetivo é claro:** Precisamos de Observabilidade Total e Resposta Ativa. Se um serviço ficar lento, precisamos ver o Trace; se der erro, precisamos dos Logs centralizados; se um limite for atingido, o time deve ser acordado pelo PagerDuty/OpsGenie; e o próprio sistema deve tentar se curar (Self-Healing).

## PROPOSTA

Sua missão é instrumentar os microsserviços do ToggleMaster, implementar uma stack de monitoramento Opensource padronizada com OpenTelemetry no Kubernetes, integrar uma solução de APM para visibilidade profunda e configurar a automação de resposta e mitigação de incidentes.

### Requisitos técnicos

#### 1. Monitoramento Opensource (Métricas e Logs no K8s)

*(Disciplinas: Prometheus, Grafana, Loki, Observabilidade no Kubernetes)*

Provisione as ferramentas base de observabilidade dentro do seu cluster Kubernetes (adicionando os Helm charts ao seu repositório GitOps):

- **Prometheus:** para armazenamento e consulta de métricas de infraestrutura.
- **Loki:** para centralização e indexação de todos os logs dos contêineres do cluster.
- **Grafana:** como ferramenta principal de visualização.
  - Crie, no mínimo, **1 Dashboard customizado** no Grafana que centralize a saúde do ecossistema: uso de recursos do cluster, taxa de requisições dos microsserviços e painel de logs em tempo real.

## 2. OpenTelemetry (OTel) e Padronização

*(Disciplinas: OpenTelemetry (OTel) e Alloy)*

O monitoramento moderno exige padronização.

- **OTel Collector (Obrigatório):** utilize o OpenTelemetry (via OTel Collector) como a peça central para receber, processar e exportar suas métricas, logs e traces. Os microsserviços devem enviar os dados de telemetria para o OTel, e ele se encarregará de roteá-los para os backends corretos (Prometheus, Loki e APM).

***Nota: O uso do Grafana Alloy no lugar do OTel Collector padrão é opcional.***

## 3. Instrumentação e APM (Traces e Visibilidade Profunda)

*(Disciplinas: APM, Instrumentação, Datadog ou New Relic)*

- **Ferramenta APM:** escolha **UMA** ferramenta comercial abordada no curso (**Datadog** OU **New Relic** - use as contas *gratuitas for education*).
- **Instrumentação do Código:** altere o código-fonte ou o Dockerfile dos microsserviços para adicionar as bibliotecas de instrumentação (preferencialmente utilizando as bibliotecas do OpenTelemetry para a linguagem específica).
- **Distributed Tracing:** gere rastreamentos distribuídos. Ao fazer uma requisição no evaluation-service, o APM deve mostrar o "caminho" da requisição.
- **Service Map:** o painel do APM deve exibir o mapa de dependências dos seus 5 microsserviços.

## 4. Alertas Inteligentes e Self-Healing

*(Disciplinas: Alert Fatigue, PagerDuty/OpsGenie, ChatOps, Runbook Automation, Self-Healing)*

- **Criação do Alerta:** configure um alerta inteligente (no Grafana ou APM). Exemplo: Disparar se a taxa de erros HTTP 5xx do auth-service for maior que 5%.
- **Gerenciamento de Incidentes:** crie uma conta no **PagerDuty** OU **OpsGenie** e integre o alerta para abrir um incidente automaticamente.
- **Notificação (ChatOps):** configure o envio de uma notificação detalhada para um canal do **Slack**, **Discord** ou **Teams**.
- **Self-Healing (Obrigatório):** crie uma automação (Runbook Automation, AWS Lambda, ou GitHub Action disparada via webhook) que reaja automaticamente ao alerta criado. Exemplo: se o alerta disparar, o script deve executar um `kubectl rollout restart deployment/auth-service` para tentar mitigar a falha sem intervenção humana.

#### Entregáveis da Fase 4

**REGRA DE AVALIAÇÃO:** É **obrigatório** evidenciar o funcionamento de **todas** as métricas, configurações e requisitos solicitados abaixo. Qualquer requisito que não for claramente demonstrado no vídeo ou documentado no relatório **sofrerá dedução direta de pontos**. Não basta configurar; é preciso mostrar operando na prática.

#### Código Fonte no Repositório:

- Código IaC/GitOps atualizado para incluir a stack de monitoramento e o OTel Collector.
- Código fonte das aplicações atualizado com a instrumentação.
- Scripts ou automações criadas para o Self-Healing.

#### Vídeo de Demonstração (até 25 min):

- **Visão Geral:** mostre que o cluster EKS (IaC) e o ArgoCD (GitOps) continuam rodando com a nova stack.
- **Métricas, Logs e OTel:** abra o Grafana. Mostre seu dashboard customizado exibindo as métricas. Busque logs no Loki. Explique rapidamente como o OTel Collector está roteando esses dados.

- **APM e Tracing:** abra o painel do Datadog/New Relic. Mostre o "Service Map". Faça uma requisição de teste e exiba o **Trace Distribuído** detalhado no APM.
- **Incidente e Self-Healing (A Prova Real):**
  - Cause um problema proposital no sistema (ex: pare o banco de dados do flag-service ou injete erro no código).
  - Mostre o alerta disparando (Firing) no Grafana/APM.
  - Mostre o incidente sendo aberto no PagerDuty/OpsGenie e a notificação chegando no Slack/Discord.
  - **Mostre a automação de Self-Healing sendo acionada e executando a ação corretiva** (ex: mostrando o pod reiniciando automaticamente via terminal).

**Relatório de Entrega (.PDF):**

- Nomes, RMs e usernames.
- Links do repositório de código e do vídeo.
- **Evidências Visuais (Obrigatório):**
  - Print do Dashboard do Grafana.
  - Print de um Trace distribuído no APM.
  - Print da notificação de incidente no ChatOps.
  - Print do log/execução da automação de Self-Healing.
- **Justificativa Técnica:** breve explicação da arquitetura do OTel implementada e da escolha entre Datadog vs New Relic e OpsGenie vs PagerDuty.

Lembrando que qualquer dúvida, podem nos chamar no Discord.

**Boa Sorte!**

The background is a dark, textured surface with a network of glowing teal lines and small dots, resembling a digital or data visualization. The lines are thin and vertical, with some horizontal connections, creating a sense of connectivity and flow. The dots are small and bright, scattered along the lines and in the open spaces. The overall aesthetic is futuristic and technological.

POS TECH